

NAME - Shreya N B

USN - 2VX24UE051

EXPT.NO - 10

TITLE - Hamiltonian Cycles in Directed Graph using Backtracking

```
In [10]: 1 n = int(input("Enter number of vertices in the graph: "))
2 print("Enter the adjacency matrix:")
3 graph = [list(map(int, input().split())) for _ in range(n)]
4
5 def find_hamiltonian_cycles(graph):
6     n = len(graph)
7     path = [0]
8     visited = [1] + [0] * (n - 1)
9     cycles = []
10
11     def backtrack(v):
12         # If all vertices are included and there is an edge back to start
13         if len(path) == n and graph[v][0]:
14             cycles.append(path + [0])
15             return
16
17         for u in range(n):
18             # For directed graph, follow only directed edges
19             if graph[v][u] == 1 and not visited[u]:
20                 visited[u] = 1
21                 path.append(u)
22
23                 backtrack(u)
24
25                 path.pop()
26                 visited[u] = 0
27
28     backtrack(0)
29     return cycles
30
31
32 print("=====")
33 cycles = find_hamiltonian_cycles(graph)
34
35 if cycles:
36     print("Hamiltonian Cycles found:")
37     for cycle in cycles:
38         print(" -> ".join(map(str, cycle)))
39 else:
40     print("No Hamiltonian Cycle found")
41
42 print("=====")
```

Enter number of vertices in the graph: 7

Enter the adjacency matrix:

```
0 1 1 0 0 1 0
1 0 1 1 0 1 0
1 1 0 1 0 1 1
0 1 1 0 1 0 1
0 0 0 1 0 0 1
1 1 1 0 0 0 1
0 0 0 1 1 1 0
```

=====

Hamiltonian Cycles found:

```
0 -> 1 -> 2 -> 3 -> 4 -> 6 -> 5 -> 0
0 -> 1 -> 3 -> 4 -> 6 -> 5 -> 2 -> 0
0 -> 1 -> 5 -> 6 -> 4 -> 3 -> 2 -> 0
0 -> 2 -> 1 -> 3 -> 4 -> 6 -> 5 -> 0
0 -> 2 -> 3 -> 4 -> 6 -> 5 -> 1 -> 0
0 -> 2 -> 5 -> 6 -> 4 -> 3 -> 1 -> 0
0 -> 2 -> 6 -> 4 -> 3 -> 1 -> 5 -> 0
0 -> 5 -> 2 -> 6 -> 4 -> 3 -> 1 -> 0
0 -> 5 -> 6 -> 4 -> 3 -> 1 -> 2 -> 0
0 -> 5 -> 6 -> 4 -> 3 -> 2 -> 1 -> 0
```

=====

```

In [1]: 1 n = int(input("Enter number of vertices in the graph: "))
2 print("Enter the adjacency matrix:")
3 graph = [list(map(int, input().split())) for _ in range(n)]
4
5 def find_hamiltonian_cycles(graph):
6     n = len(graph)
7     path = [0]
8     visited = [1] + [0] * (n - 1)
9     cycles = []
10
11     def backtrack(v):
12         # If all vertices are included and there is an edge back to start
13         if len(path) == n and graph[v][0]:
14             cycles.append(path + [0])
15             return
16
17         for u in range(n):
18             # For directed graph, follow only directed edges
19             if graph[v][u] == 1 and not visited[u]:
20                 visited[u] = 1
21                 path.append(u)
22
23                 backtrack(u)
24
25                 path.pop()
26                 visited[u] = 0
27
28     backtrack(0)
29     return cycles
30
31
32 print("=====")
33 cycles = find_hamiltonian_cycles(graph)
34
35 if cycles:
36     print("Hamiltonian Cycles found:")
37     for cycle in cycles:
38         print(" -> ".join(map(str, cycle)))
39 else:
40     print("No Hamiltonian Cycle found")
41
42 print("=====")

```

Enter number of vertices in the graph: 4

Enter the adjacency matrix:

0 1 0 0

0 0 1 1

0 1 0 1

1 0 1 0

=====
Hamiltonian Cycles found:

0 -> 1 -> 2 -> 3 -> 0

=====

```

In [2]: 1 n = int(input("Enter number of vertices in the graph: "))
2 print("Enter the adjacency matrix:")
3 graph = [list(map(int, input().split())) for _ in range(n)]
4
5 def find_hamiltonian_cycles(graph):
6     n = len(graph)
7     path = [0]
8     visited = [1] + [0] * (n - 1)
9     cycles = []
10
11     def backtrack(v):
12         # If all vertices are included and there is an edge back to start
13         if len(path) == n and graph[v][0]:
14             cycles.append(path + [0])
15             return
16
17         for u in range(n):
18             # For directed graph, follow only directed edges
19             if graph[v][u] == 1 and not visited[u]:
20                 visited[u] = 1
21                 path.append(u)
22
23                 backtrack(u)
24
25                 path.pop()
26                 visited[u] = 0
27
28     backtrack(0)
29     return cycles
30
31
32 print("=====")
33 cycles = find_hamiltonian_cycles(graph)
34
35 if cycles:
36     print("Hamiltonian Cycles found:")
37     for cycle in cycles:
38         print(" -> ".join(map(str, cycle)))
39 else:
40     print("No Hamiltonian Cycle found")
41
42 print("=====")

```

Enter number of vertices in the graph: 5

Enter the adjacency matrix:

0 1 0 1 0

1 0 1 1 1

0 1 0 0 1

1 1 0 0 1

0 1 1 1 0

=====
Hamiltonian Cycles found:

0 -> 1 -> 2 -> 4 -> 3 -> 0

0 -> 3 -> 4 -> 2 -> 1 -> 0

=====

```

In [3]: 1 n = int(input("Enter number of vertices in the graph: "))
2 print("Enter the adjacency matrix:")
3 graph = [list(map(int, input().split())) for _ in range(n)]
4
5 def find_hamiltonian_cycles(graph):
6     n = len(graph)
7     path = [0]
8     visited = [1] + [0] * (n - 1)
9     cycles = []
10
11     def backtrack(v):
12         # If all vertices are included and there is an edge back to start
13         if len(path) == n and graph[v][0]:
14             cycles.append(path + [0])
15             return
16
17         for u in range(n):
18             # For directed graph, follow only directed edges
19             if graph[v][u] == 1 and not visited[u]:
20                 visited[u] = 1
21                 path.append(u)
22
23                 backtrack(u)
24
25                 path.pop()
26                 visited[u] = 0
27
28     backtrack(0)
29     return cycles
30
31
32 print("=====")
33 cycles = find_hamiltonian_cycles(graph)
34
35 if cycles:
36     print("Hamiltonian Cycles found:")
37     for cycle in cycles:
38         print(" -> ".join(map(str, cycle)))
39 else:
40     print("No Hamiltonian Cycle found")
41
42 print("=====")

```

Enter number of vertices in the graph: 3

Enter the adjacency matrix:

0 0 1

0 0 1

1 0 0

=====

No Hamiltonian Cycle found

=====

```

In [4]: 1 n = int(input("Enter number of vertices in the graph: "))
2 print("Enter the adjacency matrix:")
3 graph = [list(map(int, input().split())) for _ in range(n)]
4
5 def find_hamiltonian_cycles(graph):
6     n = len(graph)
7     path = [0]
8     visited = [1] + [0] * (n - 1)
9     cycles = []
10
11     def backtrack(v):
12         # If all vertices are included and there is an edge back to start
13         if len(path) == n and graph[v][0]:
14             cycles.append(path + [0])
15             return
16
17         for u in range(n):
18             # For directed graph, follow only directed edges
19             if graph[v][u] == 1 and not visited[u]:
20                 visited[u] = 1
21                 path.append(u)
22
23                 backtrack(u)
24
25                 path.pop()
26                 visited[u] = 0
27
28     backtrack(0)
29     return cycles
30
31
32 print("=====")
33 cycles = find_hamiltonian_cycles(graph)
34
35 if cycles:
36     print("Hamiltonian Cycles found:")
37     for cycle in cycles:
38         print(" -> ".join(map(str, cycle)))
39 else:
40     print("No Hamiltonian Cycle found")
41
42 print("=====")

```

Enter number of vertices in the graph: 8

Enter the adjacency matrix:

```

0 1 1 0 1 0 0 0
0 0 0 1 1 0 0 0
0 1 0 1 0 0 1 0
0 0 1 0 0 0 1 0
1 0 0 0 0 1 0 0
0 0 0 0 0 0 1 1
  0 0 0 0 0 1 1 0
0 0 1 0 0 0 0 1

```

=====
No Hamiltonian Cycle found
=====

```

In [7]: 1 n = int(input("Enter number of vertices in the graph: "))
2 print("Enter the adjacency matrix:")
3 graph = [list(map(int, input().split())) for _ in range(n)]
4
5 def find_hamiltonian_cycles(graph):
6     n = len(graph)
7     path = [0]
8     visited = [1] + [0] * (n - 1)
9     cycles = []
10
11     def backtrack(v):
12         # If all vertices are included and there is an edge back to start
13         if len(path) == n and graph[v][0]:
14             cycles.append(path + [0])
15             return
16
17         for u in range(n):
18             # For directed graph, follow only directed edges
19             if graph[v][u] == 1 and not visited[u]:
20                 visited[u] = 1
21                 path.append(u)
22
23                 backtrack(u)
24
25                 path.pop()
26                 visited[u] = 0
27
28     backtrack(0)
29     return cycles
30
31
32 print("=====")
33 cycles = find_hamiltonian_cycles(graph)
34
35 if cycles:
36     print("Hamiltonian Cycles found:")
37     for cycle in cycles:
38         print(" -> ".join(map(str, cycle)))
39 else:
40     print("No Hamiltonian Cycle found")
41
42 print("=====")

```

Enter number of vertices in the graph: 5

Enter the adjacency matrix:

```

0 1 0 1 0
1 0 1 0 1
1 1 0 1 1
1 0 1 0 1
0 1 1 1 0

```

=====
Hamiltonian Cycles found:

```

0 -> 1 -> 2 -> 4 -> 3 -> 0
0 -> 1 -> 4 -> 2 -> 3 -> 0
0 -> 1 -> 4 -> 3 -> 2 -> 0
0 -> 3 -> 2 -> 4 -> 1 -> 0
0 -> 3 -> 4 -> 1 -> 2 -> 0
0 -> 3 -> 4 -> 2 -> 1 -> 0

```

=====

In []:

1